Лабораторная работа №4
«Криптографические средства защиты информации»

Выполнила Авдонина Лада, гр. 23202
Дата оформления отчета: 25.10.2026

Задание №1. Программная реализация поточного шифра

Алгоритм – RC4.
RC4 - это поточный симметричный шифр. Он генерирует псевдослучайный ключевой поток (keystream) той же длины, что и открытый текст, и комбинирует его с данными с помощью операции исключающего ИЛИ (XOR). Алгоритм состоит из двух основных этапов:

1. KSA (Key Scheduling Algorithm) - инициализация внутреннего состояния.

Создается массив S из 256 байт, изначально заполненный значениями от 0 до 255. Затем массив перемешивается с использованием секретного ключа. Для каждого индекса i вычисляется значение j по формуле:
`j = (j + S[i] + key[i mod длина_ключа]) mod 256`
После этого элементы S[i] и S[j] меняются местами. В результате получается перемешанный S-блок, зависящий от ключа.

2. PRGA (Pseudo-Random Generation Algorithm) - генерация ключевого потока.

Два индекса i и j обнуляются. Для каждого байта открытого текста выполняются следующие действия:
`i = (i + 1) mod 256`
`j = (j + S[i]) mod 256`
элементы S[i] и S[j] меняются местами
байт ключевого потока `k = S[(S[i] + S[j]) mod 256]`
шифротекст = открытый_текст XOR k
Важной особенностью RC4 является то, что одна и та же процедура используется как для шифрования, так и для расшифрования. Это возможно благодаря свойству операции XOR: повторное применение XOR с одним и тем же ключевым потоком возвращает исходные данные.
Программа написана на языке Python. Пользователь вводит с клавиатуры открытый текст и ключ. Текст и ключ преобразуются в байтовые последовательности (кодировка UTF-8). Затем выполняется шифрование и расшифрование с помощью одной и той же функции rc4(). Результат шифрования выводится в шестнадцатеричном виде. После расшифрования выполняется проверка совпадения исходного и полученного текста.
Результаты работы программы:

```
a1234@MacBook-Air-8 lab4 % python3 task1_rc4.
py
Введите строку: привет
Введите ключ: секрет

Исходная строка: привет
Ключ: секрет
Шифртекст (hex): 8ac6059e0a2ab8b427314ee5
Расшифрованная строка: привет
Проверка: успешно
```

Исходный код программы в Приложении №1.

Задание №2. Программная реализация блочного шифра

Алгоритм - RC5.
RC5 - это блочный симметричный шифр. В данной реализации используется размер блока 64 бита (8 байт), 32-битные слова и 12 раундов шифрования. Алгоритм работает с двумя 32-битными половинками блока (left и right) и выполняет над ними последовательность преобразований с использованием подключей, полученных из секретного ключа. Алгоритм состоит из следующих основных этапов:

1. Расширение ключа (Key Expansion)

Из ключа пользователя формируется массив подключей. Сначала ключ преобразуется в массив 32-битных слов. Затем создается массив подключей, который инициализируется с помощью специальных констант (0xB7E15163 и 0x9E3779B9). После этого выполняется перемешивание подключей с данными ключа с использованием циклических сдвигов. В результате получается набор подключей, который используется в раундах шифрования и расшифрования.

2. Шифрование блока

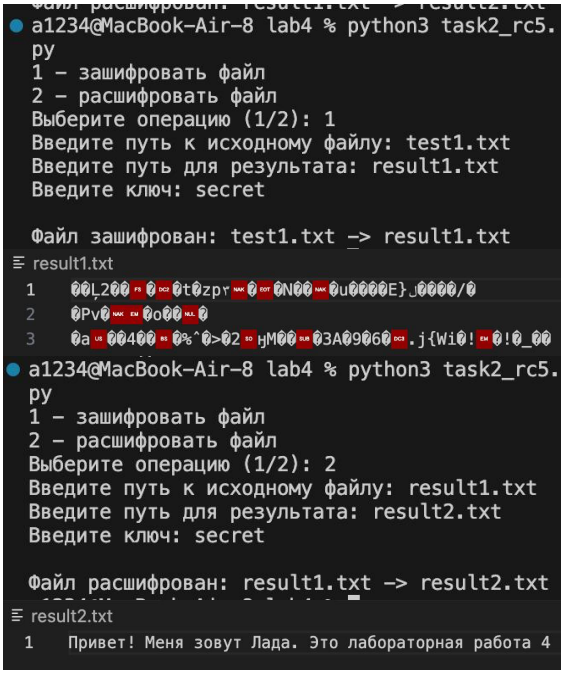
Блок данных (8 байт) делится на две 32-битные половины. К ним прибавляются первые два подключа. Далее выполняется 12 раундов, в каждом из которых происходят операции исключающего ИЛИ (XOR), циклического сдвига влево и сложения с соответствующим подключом. После всех раундов получается зашифрованный блок.

3. Расшифрование блока

Выполняются те же операции, но в обратном порядке: вычитание подключей, циклический сдвиг вправо и XOR.
Для шифрования файлов произвольной длины используется режим сцепления блоков CBC (Cipher Block Chaining). Перед шифрованием каждого блока он комбинируется с помощью XOR с предыдущим зашифрованным блоком (или с вектором инициализации IV для первого блока). Вектор инициализации генерируется случайным образом и записывается в начало зашифрованного файла.
Также применяется дополнение данных по схеме PKCS#7, чтобы длина файла была кратна размеру блока (8 байт). После расшифрования дополнение удаляется.
Программа написана на языке Python. Пользователь выбирает режим работы (шифрование или расшифрование файла), указывает пути к исходному и результирующему файлам, а также вводит ключ. Программа выполняет соответствующую операцию с использованием алгоритма RC5 в режиме CBC.
Результаты работы программы:

test1.txt

```
1    Привет! Меня зовут Лада. Это лабораторная работа 4
```



Исходный код программы в Приложении №2.

Задание №3. Хеш-функция на основе блочного шифра и проверка лавинного эффекта

На основе блочного шифра RC5 из задания 2 была реализована простая хеш-функция по схеме Дэвиса-Мейера (Davies-Meyer).

В схеме Дэвиса-Мейера хеш-значение вычисляется итеративно. На каждом шаге выполняется следующее преобразование:

`H_i = RC5_encrypt(ключ = блок_сообщения, данные = H_{i-1}) ⊕ H_{i-1}`

То есть текущий блок сообщения используется в качестве ключа для шифрования предыдущего значения хеша, после чего результат комбинируется с предыдущим значением хеша с помощью операции XOR.

Перед обработкой сообщение дополняется: добавляется байт 0x80, затем необходимое количество нулевых байтов, а в конец записывается длина исходного сообщения в битах. Это позволяет однозначно восстанавливать длину сообщения и обрабатывать данные любой длины. В качестве начального значения хеша используется фиксированный вектор Lab4Hash.

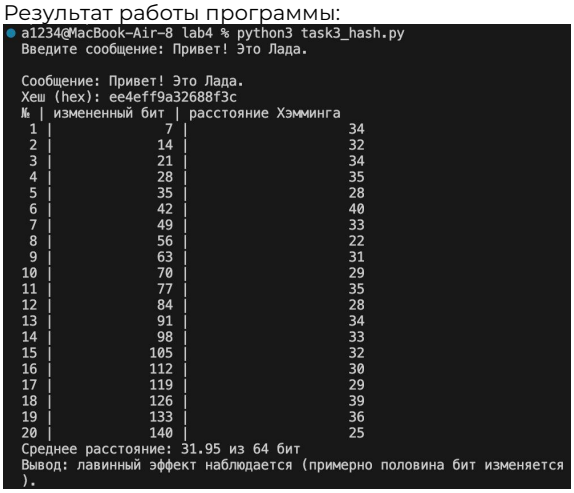
Проверка лавинного эффекта

Лавинный эффект - важное свойство криптографических хеш-функций: при изменении всего одного бита входного сообщения должно изменяться примерно половина битов выходного хеша.

Методика проверки:

1. Вычисляется хеш исходного сообщения H1.
2. В сообщении инвертируется ровно один бит.
3. Вычисляется хеш измененного сообщения H2.
4. Подсчитывается расстояние Хэмминга между H1 и H2 (количество различающихся бит).
5. Эксперимент повторяется 20 раз для разных позиций изменяемого бита.
6. Вычисляется среднее значение расстояния Хэмминга.

Результат работы программы:



Вывод: Среднее количество изменившихся бит составило 31.95, что очень близко к 32 (половине длины хеша). Это свидетельствует о наличии выраженного лавинного эффекта. Таким образом, реализованная хеш-функция на основе RC5 удовлетворяет одному из важных требований, предъявляемых к криптографическим хеш-функциям.

Исходный код программы в Приложении №3.

Задание №4. Работа с GnuPG

1. **Генерация пары ключей**

```
a1234@MacBook-Air-8 ~ % gpg --full-generate-key
gpg (GnuPG) 2.5.24; Copyright (C) 2026 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
  (16) ECC and Kyber
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Lada
Адрес электронной почты: l.avdonina@g.nsu.ru
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "Lada <l.avdonina@g.nsu.ru>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? o
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /Users/a1234/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/Users/a1234/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/Users/a1234/.gnupg/openpgp-revocs.d/B4EF1B03983347077E50D1BF1B709EA22AAF9F4C.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2026-10-27 [SC]
      B4EF1B03983347077E50D1BF1B709EA22AAF9F4C
uid                      Lada <l.avdonina@g.nsu.ru>
sub   rsa4096 2026-10-27 [E]
      1C327637623CDF1FCF639EC18E0790A7D62FBE62
```

Проверка, что ключ появился:

```
a1234@MacBook-Air-8 ~ % gpg --list-keys
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
pub   rsa4096 2026-10-27 [SC]
      B4EF1B03983347077E50D1BF1B709EA22AAF9F4C
uid         [  абсолютно ] Lada <l.avdonina@g.nsu.ru>
sub   rsa4096 2026-10-27 [E]
      1C327637623CDF1FCF639EC18E0790A7D62FBE62
```

2. Экспорт открытого ключа в файл .asc

```
a1234@MacBook-Air-8 ~ % gpg --export --armor --output public_key.asc l.avdonina@g.nsu.ru
a1234@MacBook-Air-8 ~ % ls -l public_key.asc
-rw-r--r--  1 a1234  staff  3204 Oct 27 22:30 public_key.asc
a1234@MacBook-Air-8 ~ % cat public_key.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGrgwjsBEADYp5EgS08LPEBjFj/7YZH2hk32Kp9Ef7fUti7irx7oUYXFvo7h
L+Aomg0XGRd3rbQ0YuiKMM9NEIs8AeT326ScuIRli/OAZXQyVgAwuZkCxa/Q+zyF
ZuBOaq9wmYTaVGpx098Ksyje+znLaRWR9aC0qPC6Od+QaZm3JCZS+x46CN6V4DfK
NieCcnwMgKG1DHjW5UIpzaJaDMXY0RMlxs3dxJTiIfUOhVUqs2nlE/gSua31MMB9
8dwoilKHmOZIzDQZFZxWDo9E0VsPuOtlzADnJQ3hCUN4Rc9J8GRInQy91TK/oce7
jwweWde+treVTIwTHd8DUlE7sXYCOUq6knufzh+obmGooBAZsWKeGHt68moqN0Fa
PT2yIo6W+ZTJVPkO4xQAR5CNvDIQ8HUu0np2hol9an7OQ8NSn9FkZdF8P9EwQery
ymzz0VKfqqcGhh5P5AWVvQjUreIhhwu+vz5gaqmAgtzhwblaGT4fM6msDPeh6bdg
9rSzN59RfFYgtIWLPHpx+ujczrDmc1y+bOSycC3d/0Y1AWTGLC/NgyseVQ3VIjDN
tGgqJd9HlM98Lzwi56NxkcvqxONVvBuo87JJ80nUYhZQMDoBhdoNSTqdAbroZp2F
FKTMBB6Z1/oalMyr1sRRRY7W3bAXVvMMdWAJ9uE9L66EhOvlNqE4fy55VQARAQAB
tBpMYWRhIDxsLmF2ZG9uaW5hQGcubnN1LnJ1PokCbQQTAQgAVxYhBLTvGwOYM0cH
flDRvxtwnqIqr59MBQJq4MI7GxSAAAAAAAQADm1hbnUyLDIuNSsxLjEyLDAsMwIb
AwULCQgHAgIiAgYVCgkICwIEFgIDAQIeBwIXgAAKCRAbcJ6iKq+fTB7REAC5KPxp
JPZcvxrEG7vAvEIm3TTaUXXUemvkW6R79YUTub12PVDsxRe0Wfeyh6uRaieVN5JK
Dkt30C/O5yVndZQNkbbi3YvMaGZqLtuUd16xo1gJks5Ct/xksCt2VrRBxdwxVuUj
ljAsq0xvBUylWGdx7KR/v5ZsC3llz7m1y1zJVSQd2buMv1wRgflHKSMj8QOSlPhC
ixo//x3sMtHaHcO9ceVx6PCHyaH4cydbFitCcJ27Ql+xdyUSaCaU5+zkPitmAS8/
Fx7P6llnKhWjWaxqKdyPm+AIrNKJVnBwrYR9+0czxuL2kCKqNbKkfMzSf2oBf88U
dJinumRNDQM8EJA9zww5C2SwEoMgF7XejEjmDQgeLzuypMApez76i61whF+wOStt
EqSjFuQ638uKPDi9AgT70s7zkFm1j66cq1J4oHU0atsPjqqpuqVKCMh2iH3OxMSb
sASM2tB4Fd2ACVvUO98O5J06C8V9uAs9wqjMkNbYawwTzsxyNwSyYLBcXKkWeTpp
/8KKHgW3hcMev7sziTlYj6gqQ1Zml3+8W1zxFOLwIkhvTUInGm6tACz9f4wCVqxX
7hhdhpTy3GHj+87kqUQV7kS3oEvQFvdk45vFZb/UaKhP4XQbwwzP/ZWtkzaJV8gO
+TEOa8Noxqc26rORis5QHxXgZAcM8Ses5LkaCLkCDQRq4MI7ARAA5zRv8OiGMXMJ
U40U+V5WIX3CT00D31kvhN1+Jw1LsluBft1CzljFeDgh7RaCqSxczoxagVK7tLOy
gdsO+dbOW9HgdU+5Bt4qFkd8TiIK1aTrxz+0uIzAJxqDKumg/bnxjTVGdTnCDKrr
hDoGbrYzNw4srZsfpiWPK7ovn1V1q1HJWNk0xHQzPM5U7yVn2BNVnrmHy15qYTEv
1JLnkMoq3TvwkTmSs5eVRQ/T0GvZ/meANVpQzKadqHuYWdwX8UdNsnDjvRPEIa/0
tHaLJ7Fsw5CBSHf40GFvNk7hAShLGsyNhsC70QM8XylcQvwJqfPlS0PpaO1eWKNk
WvOpb7aHm8r/wMxmFolJ4uqc/fjpklB1ayMZcyrH6W8UAs0uOdwA68CnWRR7D02k
mP2f8d0sRkVyWisswHa3KGPFxzYpnjLqe6d6lzWiMHvTdqbXdf6ZvQF6dlW83wEr
WWBstFh5Yl U7Kfk//dZoOryqTkQbMxQfOHrOafXn5V3+FJs35QVfcLnvqEY2eI8+
auVDk/QU0xZSaXljQpotpyKRjg0DL+aRu/XqyK7bfSl721OC/2K98z6CGQ0oVodI
b3clAtj7r2bDQS9hDyDWhIbTRTailBB73u3SctBpLCf5f7HqQP3bYUE7vVc/qw0/
xbqe+aPBPs+6lgjfFaarFr5sgF3HDm8AEQEAAYkCUgQYAQgAPBYhBLTvGwOYM0cH
flDRvxtwnqIqr59MBQJq4MI7GxSAAAAAAAQADm1hbnUyLDIuNSsxLjEyLDAsMwIb
DAAKCRAbcJ6iKq+fTNAbEAC14U7+Pai5fEybSGVMYKLbTzmBkNApzEJ1f554Y1Li
YnOa8IF7UTPIZsaPUQ3dKl/YGR+3M/mGGjNltymSHMmFrgzbanizy40Jy3m5t+VC
XD3EdQ0bIXaYA4qISUDi28mees4xhjJLdXOYpAGTfUhGsVutHto54fSy7+Qatmdl
GvzU3OWLJtcGjt/SmDCnodoBJmaUfP9E9OqgNHq2jxP7UX0vT+Gdf/dJigfe0ahZ
zC7Y4tmjqnpSyosabxRr3oKRHfrMxAKtMoDa2ymRlb70Kc5La15inne8cEty5aRZ
1C9H7LjTqV5RpJBpcJdC+mzZqZs4vbDUamc7Qxt0gf1DZKK46x/tfLp8eqeacTz4
TSze0EvkEFhmJ4dRBY0atS9nKMwAYzM6eRezqB/hHxcSI1bNhpP8kvWC1iKWzgrd
g3Z4xSYdPHM/taJ5XV1bQmY+BwJZWOVOCtEKn40mg1QwP9mDMi2hm7uiVDwmnV7f
2DWA5yBwzQHsiYez2jWFVQbn7e6ROLvCb4dt2K7YgmGh2NDukZ5CGIfsmoF0ZSOx
pnVGg4aLXI2mVq2ti6JiPYxQ+g0nC9crL35e6WQbxE3Bssv2JAIZBvfnQpoU9cKx
we3aqTyhchXvxo+CUVD8fwxSkkjenB4uB/6Oq2Cos8gDxX8VUzPbjRqKKAUJ7PPS
jA==
=+Zh7
-----END PGP PUBLIC KEY BLOCK-----
```

3. Импорт открытого ключа обратно

```
a1234@MacBook-Air-8 ~ % gpg --import public_key.asc
gpg: ключ 1B709EA22AAF9F4C: "Lada <l.avdonina@g.nsu.ru>" не изменен
gpg: Всего обработано: 1
gpg:                неизмененных: 1
```

Проверка:

```
a1234@MacBook-Air-8 ~ % gpg --list-keys
[keyboxd]
---------
pub   rsa4096 2026-10-27 [SC]
      B4EF1B03983347077E50D1BF1B709EA22AAF9F4C
uid         [  абсолютно ] Lada <l.avdonina@g.nsu.ru>
sub   rsa4096 2026-10-27 [E]
      1C327637623CDF1FCF639EC18E0790A7D62FBE62
```

4. Шифрование и расшифрование текста

Шифрование:

```
a1234@MacBook-Air-8 ~ % echo "Секретное сообщение для лабораторной работы" > message.txt
a1234@MacBook-Air-8 ~ % gpg --encrypt --recipient l.avdonina@g.nsu.ru --armor --output message.asc message.txt
a1234@MacBook-Air-8 ~ % cat message.asc
-----BEGIN PGP MESSAGE-----

hQIMA44HkKfWL75iAQ/+N/Ch7UFOw4VUsbICrqsm6bSJwTRU7IST2WzOqWtrCyHr
AtLK5PtO/TQPzkzaOiEHyL3R+f0REy5bQMRuW701OhteBWK2YWKMPP+BPo06pyKT
h+FyllmZqIS1LFveaJoUf86j3pGm2FmHY5puxPG/wzkmApYcKqujT4+27FnFVgyN
byV6JxEoNsMxAptM0pQtt9GhWyoGL85RmYdK41GWj+ThkkN56oNQ6EnS6OiIS+JO
n7Hh2WgOuRuuyFRUh6zHJuAB6AufA/fwoYIitKnLv3rgK1S8531ryQiQYRf+vw59
6cta73dFyKIQ2QWW3kGOd7A7fu6iqBivoBdwEofxgHA+fdbqLbm6om4/GUNjwptw
d972fYmyxUzD2IeY4NVqIneUMwnHOvmPSBXY/5yy9VABwuo2VhzgJWD4CjiRs3n6
BrIZEq3TR2kEJKJyKxXxtzSclvQ+PY42lGcV+DC4Sbs7E3nzphajSvShDg+frbwF
9h4Lw3C1eJ/JAcf87E2FHLWwD1C7lu8ZecJlDQCbGRit19+E1Wc7mxSIs1Hklaqf
lBoQPcgRKoZpkzkyrz/Qy5VHV8t/AwzR9ebimfMiwNuxXtuZjvI65f78D+q5c68h
aXoJn3VbbDpr5wiWx/Mjv39j/SB9tC8BBWOGFCWhtG97pDJrX6D3HgCOyW/7yTTU
mQEJAhBe9km60dola4oUmneAhsl+amfP6XRLuFxiWuUu6dNxHA8QO4JoHh3DCAn2
L7OXV1BpqrBkfTXO9ZVzqxePD2sc1MuYrWC+N0ZcTefDIoeJAVTe5E1l+vDLYe+e
DG3DzaEbkEI8pFWiJEQmKdKwRR+Y4G6lvGqRY6bEi2Q46CVNGBoOrrlmkbGfYPa/
gawoDkxQfICeoA==
=yf4H
-----END PGP MESSAGE-----
```

Расшифрование:

```
a1234@MacBook-Air-8 ~ % gpg --decrypt --output decrypted_message.txt message.asc

a1234@MacBook-Air-8 ~ % cat decrypted_message.txt
Секретное сообщение для лабораторной работы
```

5, 6. Шифрование и расшифрование файла из командной строки

```
a1234@MacBook-Air-8 ~ % gpg --encrypt --recipient l.avdonina@g.nsu.ru --output file.gpg message.txt
a1234@MacBook-Air-8 ~ % gpg --decrypt --output decrypted.txt file.gpg
gpg: encrypted with rsa4096 key, ID 8E0790A7D62FBE62, created 2026-10-27
      "Lada <l.avdonina@g.nsu.ru>"
a1234@MacBook-Air-8 ~ % cat decrypted.txt
Секретное сообщение для лабораторной работы
```

7. Создание и проверка электронной подписи

```
a1234@MacBook-Air-8 ~ % gpg --sign message.txt
a1234@MacBook-Air-8 ~ % gpg --verify message.txt.gpg
gpg: Подпись сделана Tue Oct 27 22:49:34 2026 +07
gpg:                ключом RSA с идентификатором B4EF1B03983347077E50D1BF1B709EA22AAF9F4C
gpg: Действительная подпись пользователя "Lada <l.avdonina@g.nsu.ru>" [абсолютное]
```

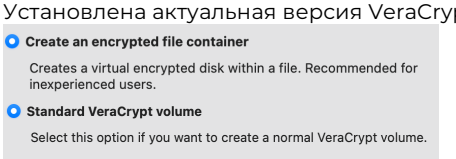
8. Безопасное удаление файла

```
a1234@MacBook-Air-8 ~ % rm -P file.gpg
```

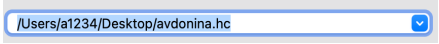
Флаг -P на macOS перезаписывает файл перед удалением.

Задание №5. Шифрование данных с помощью VeraCrypt

Установлена актуальная версия VeraCrypt с официального сайта.

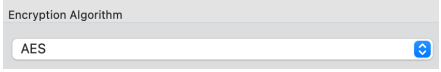


Создан файл-контейнер с именем avdonina.hc

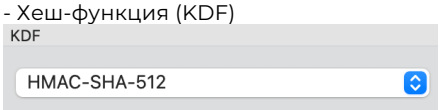


При создании контейнера были выбраны следующие параметры:

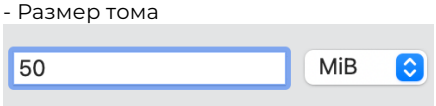
- Алгоритм шифрования



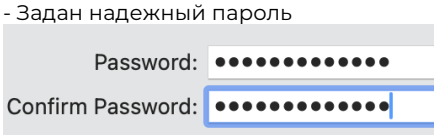
- Хеш-функция (KDF)



- Размер тома



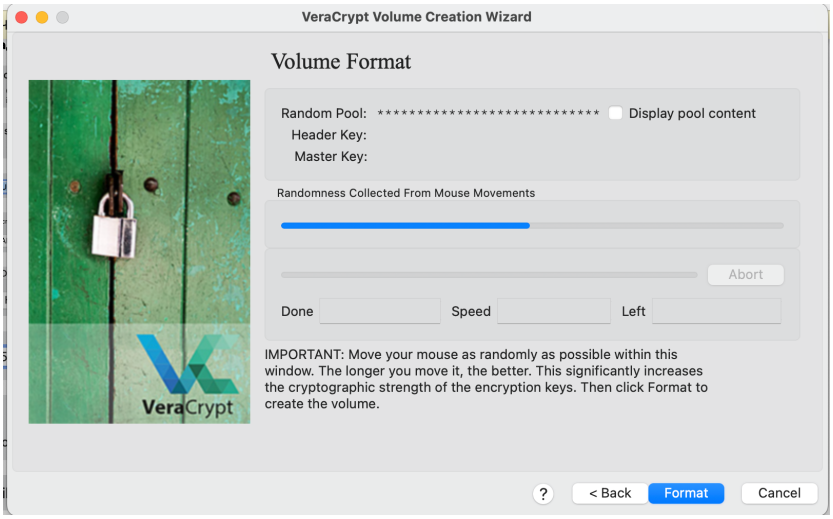
- Задан надежный пароль



- Файловая система



На этапе форматирования тома (Volume Format) программа запросила сбор энтропии путем движения мыши, после чего была нажата кнопка Format.



Причина, по которой задание выполнено не полностью:

При попытке завершить создание и смонтировать том возникла системная ошибка, связанная с компонентом macFUSE (необходим для монтирования томов VeraCrypt в macOS). Система заблокировала загрузку системного расширения macFUSE. В окне ошибки предлагалось разрешить загрузку программного обеспечения от разработчика «Benjamin Fleischer» в разделе «Конфиденциальность и безопасность». Однако в настройках macOS кнопка «Разрешить» не отображалась, из-за чего не удалось активировать необходимое расширение ядра. Без успешно загруженного macFUSE невозможно завершить форматирование тома, смонтировать созданный контейнер, скопировать в него файлы и выполнить дальнейшие действия по заданию (размонтирование, повторное монтирование и проверка доступности файлов).

ПРИЛОЖЕНИЯ

Приложение №1. Реализация алгоритма RC4

```python
#!/usr/bin/env python3

def rc4(data: bytes, key: bytes) -> bytes:
    # Поточный шифр RC4
    # Одна и та же функция используется и для шифрования, и для расшифрования,
    # потому что операция XOR обратима: (P ⊕ K) ⊕ K = P
    if not key:
        raise ValueError("Ключ не должен быть пустым.")
    # KSA (Key Scheduling Algorithm)
    # Инициализация S-блока: S[i] = i для i = 0...255
    state = list(range(256))
    j = 0
    for i in range(256):
        # Перемешивание S-блока с учетом ключа
        j = (j + state[i] + key[i % len(key)]) & 0xFF   # &0xFF == mod 256
        state[i], state[j] = state[j], state[i]         # обмен элементов
    # PRGA (Pseudo-Random Generation Algorithm)
    # Генерация ключевого потока и XOR с данными
    result = bytearray()
    i = j = 0
    for value in data:
        i = (i + 1) & 0xFF
        j = (j + state[i]) & 0xFF
        state[i], state[j] = state[j], state[i]         # снова обмен
        # Байт ключевого потока
        k = state[(state[i] + state[j]) & 0xFF]
        result.append(value ^ k)                       # XOR с исходным байтом
    return bytes(result)

def main() -> None:
    text = input("Введите строку: ")
    key_text = input("Введите ключ: ")
    source = text.encode("utf-8")
    key = key_text.encode("utf-8")
    encrypted = rc4(source, key)
    decrypted = rc4(encrypted, key).decode("utf-8")
    print(f"\nИсходная строка: {text}")
    print(f"Ключ: {key_text}")
    print(f"Шифртекст (hex): {encrypted.hex()}")
    print(f"Расшифрованная строка: {decrypted}")
    print(f"Проверка: {'успешно' if decrypted == text else 'ошибка'}")

if __name__ == "__main__":
    main()
```

Приложение №2. Реализация алгоритма RC5

```python
#!/usr/bin/env python3

import secrets
from pathlib import Path

# Константы RC5
WORD_MASK = 0xFFFFFFFF          # Маска для 32-битных слов (обрезаем до 32 бит)
ROUNDS = 12                     # Количество раундов (стандартное значение)
BLOCK_SIZE = 8                  # Размер блока = 8 байт (два слова по 4 байта)

def rotate_left(value: int, amount: int) -> int:
    # Циклический сдвиг влево на amount бит (в пределах 32 бит)
    amount &= 31                # Сдвиг больше 31 бессмысленен
    if amount == 0:
        return value
    return ((value << amount) | (value >> (32 - amount))) & WORD_MASK

def rotate_right(value: int, amount: int) -> int:
    # Циклический сдвиг вправо на amount бит
    amount &= 31
    if amount == 0:
        return value
    return ((value >> amount) | (value << (32 - amount))) & WORD_MASK

def expand_key(key: bytes) -> list[int]:
    # Расширение ключа
    # Из короткого ключа пользователя создает длинный массив подключей (S),
    # который потом используется в каждом раунде шифрования/расшифрования
    if not key:
        raise ValueError("Ключ не должен быть пустым.")
    # Преобразуем байты ключа в массив 32-битных слов
    word_count = max(1, (len(key) + 3) // 4)
    words = [0] * word_count
    for index, value in enumerate(key):
        words[index // 4] |= value << (8 * (index % 4))
    # Создаем массив подключей
    subkey_count = 2 * (ROUNDS + 1)     # Для 12 раундов нужно 26 подключей
    subkeys = [0] * subkey_count
    # Инициализация "магическими" константами
    subkeys[0] = 0xB7E15163             # Pw (для 32-бит)
    for index in range(1, subkey_count):
        subkeys[index] = (subkeys[index - 1] + 0x9E3779B9) & WORD_MASK  # Qw
    # Перемешивание подключей с ключом пользователя
    a = b = i = j = 0
    for _ in range(3 * max(word_count, subkey_count)):
        a = subkeys[i] = rotate_left((subkeys[i] + a + b) & WORD_MASK, 3)
        b = words[j] = rotate_left(
            (words[j] + a + b) & WORD_MASK, (a + b) & 31
        )
        i = (i + 1) % subkey_count
        j = (j + 1) % word_count
    return subkeys

def encrypt_block(block: bytes, key: bytes) -> bytes:
    # Шифрование одного 8-байтового блока
    subkeys = expand_key(key)
```

```
    # Делим блок на две 32-битные половинки
    left = int.from_bytes(block[:4], "little")
    right = int.from_bytes(block[4:], "little")
    # Начальное добавление подключей
    left = (left + subkeys[0]) & WORD_MASK
    right = (right + subkeys[1]) & WORD_MASK
    # 12 раундов основного преобразования
    for round_number in range(1, ROUNDS + 1):
        left = (rotate_left(left ^ right, right) + subkeys[2 * round_number]) & WORD_MASK
        right = (rotate_left(right ^ left, left) + subkeys[2 * round_number + 1]) & WORD_MASK
    return left.to_bytes(4, "little") + right.to_bytes(4, "little")

def decrypt_block(block: bytes, key: bytes) -> bytes:
    # Расшифрование одного 8-байтового блока (операции в обратном порядке)
    subkeys = expand_key(key)
    left = int.from_bytes(block[:4], "little")
    right = int.from_bytes(block[4:], "little")
    # Раунды в обратном порядке
    for round_number in range(ROUNDS, 0, -1):
        right = rotate_right((right - subkeys[2 * round_number + 1]) & WORD_MASK, left) ^ left
        left = rotate_right((left - subkeys[2 * round_number]) & WORD_MASK, right) ^ right
    # Вычитаем начальные подключи
    left = (left - subkeys[0]) & WORD_MASK
    right = (right - subkeys[1]) & WORD_MASK
    return left.to_bytes(4, "little") + right.to_bytes(4, "little")

def pad(data: bytes) -> bytes:
    # Дополнение PKCS#7
    # Добавляет в конец столько байт, чтобы длина стала кратна 8
    # Каждый добавленный байт равен количеству добавленных байт
    count = BLOCK_SIZE - len(data) % BLOCK_SIZE
    return data + bytes([count]) * count

def unpad(data: bytes) -> bytes:
    # Удаление дополнения PKCS#7
    count = data[-1]
    if not 1 <= count <= BLOCK_SIZE or data[-count:] != bytes([count]) * count:
        raise ValueError("Некорректное дополнение PKCS#7.")
    return data[:-count]

def encrypt_file(source: Path, destination: Path, key: bytes) -> None:
    # Шифрование файла в режиме CBC
    # - Генерируется случайный IV (8 байт)
    # - Каждый блок XOR-ится с предыдущим шифрблоком (или с IV)
    # - Затем шифруется
    iv = secrets.token_bytes(BLOCK_SIZE)   # Случайный вектор инициализации
    previous = iv
    encrypted = bytearray(iv)              # IV пишем в начало файла
    data = pad(source.read_bytes())        # Дополняем данные до кратности 8
    for offset in range(0, len(data), BLOCK_SIZE):
        # XOR текущего блока с предыдущим шифрблоком (CBC)
        block = bytes(a ^ b for a, b in zip(data[offset:offset + BLOCK_SIZE], previous))
        previous = encrypt_block(block, key)
        encrypted.extend(previous)
    destination.write_bytes(encrypted)

def decrypt_file(source: Path, destination: Path, key: bytes) -> None:
```

```
    # Расшифрование файла в режиме CBC
    data = source.read_bytes()
    if len(data) < 16 or len(data) % BLOCK_SIZE:
        raise ValueError("Некорректный размер зашифрованного файла.")
    previous = data[:BLOCK_SIZE]           # Первый блок — это IV
    result = bytearray()
    for offset in range(BLOCK_SIZE, len(data), BLOCK_SIZE):
        current = data[offset:offset + BLOCK_SIZE]
        decrypted = decrypt_block(current, key)
        # XOR с предыдущим шифрблоком, чтобы получить исходный блок
        result.extend(bytes(a ^ b for a, b in zip(decrypted, previous)))
        previous = current
    destination.write_bytes(unpad(bytes(result)))

def main() -> None:
    print("1 — зашифровать файл")
    print("2 — расшифровать файл")
    mode = input("Выберите операцию (1/2): ").strip()
    source = Path(input("Введите путь к исходному файлу: ").strip())
    destination = Path(input("Введите путь для результата: ").strip())
    key = input("Введите ключ: ").encode("utf-8")
    if mode == "1":
        encrypt_file(source, destination, key)
        print(f"\nФайл зашифрован: {source} -> {destination}")
    elif mode == "2":
        decrypt_file(source, destination, key)
        print(f"\nФайл расшифрован: {source} -> {destination}")
    else:
        raise ValueError("Нужно выбрать 1 или 2.")

if __name__ == "__main__":
    main()
```

Приложение №3. Хеш-функция на основе блочного шифра и проверка лавинного эффекта

```python
#!/usr/bin/env python3

from task2_rc5 import BLOCK_SIZE, encrypt_block

# Начальное значение хеша (Initialization Vector), 8 байт
INITIAL_VECTOR = b"Lab4Hash"

def davies_meyer(message: bytes) -> bytes:
    # Хеш-функция по схеме Дэвиса–Мейера
    # Идея: H_i = RC5_encrypt(ключ = блок_сообщения, данные = H_{i-1})  XOR  H_{i-1}
    # Сообщение дополняется так, чтобы его длина была кратна размеру блока,
    # и в конец записывается длина исходного сообщения в битах

    # Длина исходного сообщения в битах (8 байт)
    length = (len(message) * 8).to_bytes(8, "big")
    # Дополнение: добавляем байт 0x80, потом нули, потом длину
    padded = message + b"\x80"
    # Выравниваем так, чтобы после добавления длины длина стала кратна BLOCK_SIZE
    padded += b"\0" * ((BLOCK_SIZE - (len(padded) + 8) % BLOCK_SIZE) % BLOCK_SIZE)
    padded += length
    # Начинаем с фиксированного начального значения
    state = INITIAL_VECTOR
    # Обрабатываем сообщение по блокам
    for offset in range(0, len(padded), BLOCK_SIZE):
        message_block = padded[offset:offset + BLOCK_SIZE]
        # Дэвис–Мейер: шифруем текущий state, используя блок сообщения как ключ
        encrypted = encrypt_block(state, message_block)
        # XOR с предыдущим значением state
        state = bytes(a ^ b for a, b in zip(encrypted, state))
    return state   # итоговый хеш (8 байт)

def hamming_distance(first: bytes, second: bytes) -> int:
    # Расстояние Хэмминга — количество различающихся бит между двумя хешами
    return sum((a ^ b).bit_count() for a, b in zip(first, second))

def main() -> None:
    text = input("Введите сообщение: ")
    message = text.encode("utf-8")
    # Хеш исходного сообщения
    digest = davies_meyer(message)
    print(f"\nСообщение: {text}")
    print(f"Хеш (hex): {digest.hex()}")
    print("№ | измененный бит | расстояние Хэмминга")
    distances = []
    # 20 экспериментов: каждый раз меняем один бит в разных позициях
    for number in range(1, 21):
        changed = bytearray(message or b"\0")   # на случай пустого сообщения
        # Выбираем позицию бита (разные позиции в разных экспериментах)
        bit_index = (number * 7) % (len(changed) * 8)
        # Инвертируем один бит
        changed[bit_index // 8] ^= 1 << (bit_index % 8)
        # Считаем хеш измененного сообщения и расстояние Хэмминга
        distance = hamming_distance(digest, davies_meyer(bytes(changed)))
        distances.append(distance)
        print(f"{number:2d} | {bit_index:14d} | {distance:21d}")
    average = sum(distances) / len(distances)
```

```
    print(f"Среднее расстояние: {average:.2f} из 64 бит")
    # Вывод о наличии лавинного эффекта
    if 28 <= average <= 36:
        print("Вывод: лавинный эффект наблюдается (примерно половина бит изменяется).")
    else:
        print("Вывод: лавинный эффект выражен слабо или отсутствует.")

if __name__ == "__main__":
    main()
```